

IA USSAGE LOG

Herramienta	¿Para qué la utilicé?	¿Qué aportó la IA?	¿Qué verifiqué yo?
AGY	Para crear los md de los requerimientos, spec y agents.	Creación de los archivos md	La información plasmada sea correcta y refleje el objetivo de la actividad.
AGY	Entendimiento del repositorio con el MCP de Github.	Resumen del repositorio, brindando información sobre como esta contruido y también resumiendo los issues y los PR.	Que la información dada reflejara la realidad.
AGY	Reporte final	Informe sobre lo que se realizó	La información sea veridica y se hable del repositorio en el que se trabajo.

Repository Understanding

1. Propósito del Proyecto

E.L. TENDEJÓN es una aplicación de escritorio desarrollada en Java 11 y Java Swing para la gestión de punto de venta (POS) e inventario de tiendas de abarrotes.

Ofrece operaciones CRUD de artículos, cobro con cálculo de cambio y descuentos, bitácora de ventas y control de acceso por roles (Administrador y Empleado).

Utiliza persistencia local en archivos de texto plano delimitados por comas (en la carpeta registro/), sin requerir bases de datos externas.

2. Estructura y Componentes Clave

Patrón de diseño: Implementa MVC (Modelo-Vista-Controlador) junto con DAO (Data Access Object):

- **Model:** Entidades POJO (Articulo, Empleado) y excepciones de negocio (StockInsuficienteException, NumeroNegativoOCeroException, ValorDescuentoException).
- **DAO:** DAOArticulo y EmpleadoDAO, que cargan los archivos a listas en memoria (ArrayList) y reescriben los .txt en disco.
- **Control:** Controladores que implementan ActionListener para orquestar la navegación y las reglas de negocio.
- **View:** Pantallas principales y más de 15 diálogos modales construidos como JFrame independientes con posicionamiento absoluto.

- **Registro/:** Almacén físico de datos (Articulos.txt, Empleados.txt, RegistroVentas.txt, Ticket.txt).
- **Media/:** Recursos gráficos de la interfaz (íconos, logos).

3. Punto de Entrada

Ubicación: src/test/ProgramaTendjeon.java (public static void main(String[] args)). Inicia la ventana InicioSesionView y su controlador InicioSesionControl.

Aunque reside en la carpeta src/test/, está configurado explícitamente en el pom.xml como la clase principal ejecutable.

4. Estado de Pruebas

No hay pruebas automatizadas: Aunque el pom.xml incluye la dependencia junit:junit:4.13.2, en la carpeta test/ únicamente existe la clase main. El proyecto depende 100% de verificación y pruebas manuales.

5. Documentación

En main: Guías de instalación y ejecución en IDEs (SETUP.md, MAVEN_SETUP.md).

En el PR #2 (feature/Modifications): Se añade documentación formal y completa en docs/: ARCHITECTURE.md (diagramas de flujo, secuencia y arquitectura), REQUIREMENTS.md (especificación de requerimientos de software), SPEC.md (especificación técnica de clases y archivos), MCP_GITHUB_TOOL_INVENTORY.md (matriz de herramientas).

6. Hallazgos Críticos sobre el Issue #1 y PR #2

Riesgo Crítico en PR #2 ("Refactorización del idioma."): Modifica src/model/Empleado.java renombrando sus métodos a inglés (getname, getpassword), pero no actualizó las clases que lo consumen (EmpleadoDAO, InicioSesionControl, AgregarEmpleadoControl).

Consecuencia: Si se aprueba tal como está, rompe la compilación del proyecto.

Alcance de Issue #1 ("Refactorización del código"): Resolver la deuda técnica. Corregir la incompatibilidad de nombres/idioma, estandarizar rutas de archivos (reemplazar separadores de Windows \\ por multiplataforma), consolidar los más de 15 modales JFrame en componentes reutilizables y añadir pruebas unitarias reales con JUnit.

7. Capacidades MCP de GitHub Utilizadas (Solo Lectura)

- `get_file_contents`: Exploración de directorios, archivos de configuración, scripts y código fuente.
- `list_issues`: Localización del Issue #1 abierto.
- `list_pull_requests`: Detección del PR #2 abierto.
- `get_pull_request_files`: Inspección del diff exacto y cambios introducidos en el PR #2.
- `list_commits`: Trazabilidad del historial en las ramas main y feature/Modifications.

Issue Analysis

- Issue #1 Refactorización de código.
- Se solicita refactorizar el CRUD POS en Java para mejorar su estructura, legibilidad, mantenibilidad y reutilización, manteniendo el comportamiento funcional actual.

El alcance incluye:

- Eliminar código duplicado y código muerto.
- Separar responsabilidades y aplicar POO y Clean Code.
- Mejorar clases de productos, CRUD, POS, UI, validaciones y excepciones.
- Centralizar la conexión y acceso a la base de datos.
- Simplificar consultas SQL repetidas.
- Extraer y reutilizar métodos.
- Ejecutar las pruebas existentes para garantizar que la funcionalidad no cambie.
- Criterio principal: el sistema debe conservar su funcionalidad actual, compilar correctamente y mantener las pruebas pasando después del refactor.

Preguntas o ambigüedades

Se identificaron **5 puntos que requieren aclaración antes de refactorizar**:

- **Base de datos**: El proyecto no utiliza SQL ni una BD, sino archivos .txt. Se debe aclarar si se optimizará este almacenamiento o se migrará.
- **Pruebas**: El Issue menciona pruebas existentes, pero no hay pruebas automatizadas. Se debe definir si se crearán pruebas con JUnit o se realizará validación manual.
- **Nomenclatura**: El código mezcla español e inglés. Se debe establecer un idioma y una convención de nombres consistente.
- **Interfaz gráfica**: Existen muchas clases redundantes para mensajes y errores. Se debe confirmar si pueden reemplazarse por JOptionPane/JDialog reutilizables.

- **Estructura del proyecto:** Se debe aclarar si está permitido reorganizar el proyecto siguiendo la estructura estándar de Maven (src/main/java y src/test/java).

Archivos potencialmente relacionados

La refactorización involucra principalmente las siguientes áreas del proyecto:

- **Configuración:** pom.xml — compilación, empaquetado y dependencias.
- **Modelo (src/model):** entidades Artículo, Empleado y excepciones de negocio.
- **Acceso a datos (src/DAO):** DAOArticulo y EmpleadoDAO, responsables de lectura, escritura, validaciones y manejo de archivos.
- **Control (src/control):** lógica de ventas, pagos, productos y empleados. Aquí se concentra parte de la **duplicación de código y rutas dependientes de Windows**.
- **Interfaz (src/view):** ventanas principales y múltiples diálogos JFrame; destaca PagarView, que requiere corregir EXIT_ON_CLOSE.
- **Punto de entrada:** src/test/ProgramaTendejon.java, que contiene el main.
- **Datos:** archivos .txt en registro/ utilizados como almacenamiento del sistema.

Resumen de tests existentes

- **No existen pruebas automatizadas** en el repositorio.
- Aunque pom.xml incluye **JUnit 4.13.2**, no hay clases *Test.java ni métodos @Test.
- mvn test actualmente ejecuta **0 pruebas**.
- Las únicas pruebas documentadas son **manuales**, descritas en SETUP.md y MAVEN_SETUP.md, consistentes en compilar y ejecutar ProgramaTendejon.main().

Resumen de Requirements y Acceptance Criteria

La refactorización debe **mantener la funcionalidad existente** y mejorar la estructura interna sin romper los requerimientos actuales.

- **Criterios del Issue:** mantener funcionalidades, eliminar duplicación, separar responsabilidades, mejorar métodos y nombres, centralizar acceso a datos, manejar errores de forma uniforme, compilar correctamente y verificar las operaciones CRUD.
- **Requerimientos funcionales:** conservar el CRUD de inventario, operaciones de venta, cobros, cálculo de cambio y persistencia de tickets y ventas.
- **Requerimientos no funcionales:** garantizar compatibilidad multiplataforma, mantener Maven y respetar la separación **MVC + DAO**.
- **Reglas de negocio:** conservar unicidad de productos, validaciones de cantidades/precios/descuentos, protección del administrador raíz y prevención de ventas sin stock.

- **Especificación técnica:** no alterar el formato de los archivos de registro/, respetar las firmas y códigos de retorno de los DAO.
- **Deuda técnica identificada:** resolver incompatibilidades de Empleado, rutas dependientes de Windows, credenciales hardcodeadas y uso incorrecto de EXIT_ON_CLOSE.

Comprobar el boundary sin modificar GitHub

Acción consultada	¿Es posible con las herramientas expuestas?	Herramienta requerida	Estado en la sección actual
Crear issue	No	Create_issue	No expuesta
Modificar archivos del repositorio	No	Create_or_update_file	No expuesta
Comentar un pull request	Condicional	Add_issue_comment	Expuesta pero restringida por el contexto de Read-Only.